

Porting a C inference engine to Rust — and making it faster

A 28.9M-parameter language model on an \$8 microcontroller. What we got wrong on the way, and how each wrong thing got caught.

122.0 ms

C reference,
per token

95.2 ms

Rust port,
per token

28.2%

faster than C,
on
all five stages

10

wrong turns,
each
killed by a
number

ESP32-S3 @ 240 MHz · 400 tokens, 18-token prompt · byte-identical output, verified on device by token digest
0x327578cb136fd6aa

01 The starting point

A 28.9M-parameter language model runs on an ESP32-S3 — a dual-core microcontroller with 512 KB of SRAM, 8 MB of external PSRAM, and no vector unit worth the name. It fits because ~25M of its parameters live in a flash-mapped Per-Layer Embedding table and the rest are 4-bit quantized.

The implementation was a single-file C engine in the `llama2.c` style. On this board it produced a token every **122.0 ms** — 8.20 tokens per second. We set out to port it to Rust: not to rewrite it, not to redesign it, but to port it function for function and then find out whether Rust could match C on a machine this small.

A port like this has exactly one way to fail invisibly. If the Rust version computes something slightly different — a rounding mode, an accumulation order, a clamp — it will still emit plausible children's stories, and it will be faster or slower for reasons that have nothing to do with the code. So the first decision was not about performance.

THE RULE EVERYTHING ELSE RESTS ON

Everything must be bit-exact, and a machine must check it. No tolerance anywhere. Not "close enough", not "within epsilon."

That rule turned out to be the most valuable thing in the project — and not for the reason we expected.

Three ground rules

No tolerance	Golden logits against PyTorch, byte-identical CLI output against the compiled C binary, exact fp32 cross-entropy over real validation tokens. Where a change <i>could</i> alter a value, a test asserts it does not — bit for bit, on every row of the shipped model.
Measure, don't reason	Every performance claim gets a number from the actual board. A hypothesis with no measurement behind it is a guess — and this report is largely a record of how often our guesses were wrong.
Keep the failures	A change that made things worse is recorded with its number, not quietly deleted. Three are in the published benchmark table today.

02 The naive port was 2.5× slower

The first working Rust firmware ran at **311.5 ms/token**. Against C's 122.0, that is 2.55× slower — same algorithm, same model file, same board. This is the moment where a port either learns something or starts guessing.

MS / TOKEN	INPUT	ATTN	FFN	PLE	HEAD	WALL
C reference	4.4	42.9	6.9	8.5	57.1	122.0
Rust, first run	10.6	66.7	15.8	20.4	193.8	311.5

The output head — a [25353 × 96] matrix-vector product run in full for every token — was 3.4× slower than C's and 62% of the whole budget.

Three bugs wearing optimization costumes

Before any real optimization, three things were simply wrong.

BUG 1 — THE DUAL-CORE SPLIT WAS NOT DUAL-CORE

The C reference creates a worker pinned to core 0 with `xTaskCreatePinnedToCore(..., 0)`, and the port copied that faithfully. But `arduino-esp32` runs `loop()` on CPU1 while ESP-IDF pins the main task to CPU0. Same call, different scheduler context: **both halves of the head were queueing on one core**, with a task switch between them. One config line fixed it.

Bug 2 was a doubled load — the model was being read twice on the path into PSRAM. **Bug 3** was cache configuration: the external-memory caches sat at their defaults, and widening them moved the head **110.9 → 87.6 ms**, a bigger single move than anything had produced on any stage.

That last one felt like a discovery. It was the start of a wrong turn that cost several days.

03 The head, in six episodes

Episode 1 — "It's bandwidth-bound", and the change that proved nothing

Widening the caches helped enormously, so the head must be starved of memory. It streams 2.54 MB of int8 weights per token through a 64 KB data cache. The reasoning was clean.

So we packed it. The model file already stores the head as 4-bit codes; the C reference unpacks them to int8 once at boot. Staging the *packed* form halves what the stage streams — **2.54 MB** → **1.32 MB** — at the cost of a mask or shift per element, and hands back 1.2 MB of PSRAM.

RESULT

The head moved **0.0 ms**. Not "a little" — zero. And a null result like that is consistent with two opposite worlds: memory doesn't matter, or memory matters and something else is hiding it.

Episode 2 — The instrument that settled it, in one boot

Rather than argue, we built a measurement. Two of them, in the same run:

```
PSRAM read: 72 MB/s (1.22 MB of head weights => 16.9 ms/token floor)
head detail: own-half 90.9 | wait-worker 0.0 | worker-half 90.9
```

Both cores independently finished their half of the head in 90.9 ms. That is **13.4 MB/s used against 72 available — 19% utilization**. And the shape is the proof: if memory were the constraint, two cores reading at once could not *both* have hit 90.9. They would have contended.

MEASUREMENT

The head was compute-bound, and always had been. Packing bought a resource the stage was not short of — which is exactly why it measured zero. The cache-widening win that started this was real, but it was *instruction* cache; we had credited the wrong half of the same config change.

Episode 3 — The dependency chain, and a table making things slower

Compute-bound means instructions and stalls. The inner loop had a single accumulator: every multiply-accumulate waited on the one before it. Four independent accumulator chains took the head **91.0** → **81.3 ms**.

Then a sharper one. Each 4-bit code is stored biased by 8, so every element paid a -8 . But $\sum (c_j - 8) a_j = \sum c_j a_j - 8 \sum a_j$ — the bias lifts out of the loop entirely, once per row instead of once per element. **81.3** → **61.5 ms**.

A PATTERN COPIED WITHOUT RE-DERIVING IT

That change also deleted a 16-entry lookup table we had added on purpose. It was copied from the float path, where it replaces an integer → float *conversion* and is worth -24% on the host. On the integer path it replaced a *subtract* — one cycle — with a memory load.

Episode 4 — Reading the disassembly, and deleting one instruction

The head was now 62.2 ms against C's 57.1 — the last stage still behind, and stage timing had nothing left to say. So we started dumping the Xtensa both compilers emit and reading it side by side. Twelve instructions for two multiply-accumulates, two of them doing nothing:

```
l8ui  a15, a3, 0      ; load activation
sext  a15, a15, 7      ; <-- pure overhead
mull  a15, a15, a7
```

The LX7 has no signed byte load. `l8ui` zero-extends, so every `i8` activation read cost a load *and* a sign-extend — 2.43 million times per token. Widening the activation scratch to `i16` lets `l16si` sign-extend inside the load and deletes the instruction. Cost: 96 extra bytes, read once per token.

62.2 → 55.3 MS

The head crossed C for the first time — 55.3 against 57.1, 0.97× — the fifth and last stage to do so. One deleted instruction.

The same session killed two of our own claims. We had been saying the LX7's MAC16 unit was the next big lever, because the Rust binary held 814 `mull` and only 19 `mula.*`. Then we disassembled GCC's output: **zero** `mula.*`. Neither compiler uses it. And we had claimed Rust was "38% behind C on arithmetic" — a figure that assumed C's memory time was additive, which would require C to hit 6.0 cycles/MAC on an eight-instruction serial chain. Nothing in-order does that: C's single accumulator stalls, and the stalls absorb its memory latency. Both retracted in writing, in the commit that fixed the instruction.

Episode 5 — Asking whether the loop stalls, then acting on the answer

One more measurement before the next change: time the identical loop with weights resident in cache, and again streaming from PSRAM.

8.96 VS 9.64 CYCLES PER MULTIPLY-ACCUMULATE

Eight percent apart. If the loop were stalling on memory, prefetch and software pipelining would be the levers.

Eight percent apart means it is **issue-bound** — the accumulator chains already cover load-use latency, and the only thing that helps is executing fewer instructions.

Which pointed at something sitting in plain sight: the activation vector is the same 96 values for all 25,353 rows, and the loop reloaded it for every one. Two rows per pass loads each activation once and multiplies it into both.

55.3 → 49.7 MS · CYCLES/MAC 8.96 → 8.13 · 10.50 TOK/S

Close to the 11% the instruction count predicted. **Every stage now faster than C.**

Episode 6 — Two failures, kept on purpose

Four rows per pass. If two rows halve the activation loads, four should quarter them. We wrote the failure mode into the commit message before flashing: four rows need four lane sets, and at four lanes each that is sixteen live accumulators on a machine with sixteen visible address registers.

IT SPILLED — HEAD 49.3 → 52.8 MS

Cycles/MAC went the wrong way, **8.13 → 8.41** — the spill showing up directly in the one number that could name it. Reverted the same day. The arithmetic is kept, tested, off the hot path, with the measurement in its doc comment.

Staging the head as int8 again. The other direction: drop the unpack, use C's own format, one fewer operation per element. **Head 62.3 → 76.9 ms. Token 106.3 → 121.5.** Doubling the bytes cost 14.6 ms against a predicted 13.3 for the extra 1.21 MB — so the unpack arithmetic it removed was worth at most 1.3 ms across 2.43M elements. Nothing.

WHY THIS IS THE MOST USEFUL FAILURE IN THE PROJECT

It retroactively explains Episode 1. Packing measured 0.0 ms *because the single-accumulator loop was stalling, and the stalls hid the memory traffic.* Breaking the dependency chain exposed it. **The same change is worth 13 ms today and was worth 0 then.**

It also falsified what we started believing after Episode 2 — "compute-bound, so bytes don't matter." Low bandwidth utilization is not the same as insensitivity to bandwidth. Both are true at once, and only measurement tells them apart.

04 Everything that wasn't the head

While the head was 62 ms of a 138 ms token, the *other* 76 ms ran on one core with the second parked in `wait_notification`.

CHANGE	WHAT IT DOES	EFFECT
Dual-core every matvec	556k fp32 multiply-accumulates per token in <code>input</code> , <code>ffn</code> and <code>p1e</code> , split by rows. Bit-exact: <code>y[r]</code> depends only on row <code>r</code> , so which core computes it cannot change it.	-12 ms
Split attention	The matvec hook could not reach attention, because attention is not a matvec. Instrumenting it four ways found core at 28.3 ms — 24% of the token, still single-threaded. Heads divide like rows: each reads the whole KV cache, writes only its own slice.	28.3 → 15.6
Argmax into the head	Outside the profiled stages sat 2.74 ms: 25,353 floats read back from PSRAM at half bus rate, because a data-dependent branch per element leaves nothing to pipeline. Both cores already hold each logit in a register.	2.74 → 0.00

The attention split is also an honest loose end. A perfect halving plus measured sync would have been 14.25 ms, so 1.35 ms — 9.5% — did not come back, and we still cannot fully separate bus contention from code placement there. It is written down as unresolved rather than explained away.

The argmax is the one place this port is *not* doing the same work as its reference. It is doing less of it, for the same output — the C engine still pays that 2.2 ms.

05 The scoreboard of wrong ideas

WHAT WE BELIEVED	WHAT THE MEASUREMENT SAID
Attention is the worst stage, 2.10×	The two runs used different settings, and a t t n is the only stage that scales with sequence position. At matched settings it was 1.27× — the <i>closest</i> stage to parity.
The head is bandwidth-bound	19% of available bandwidth; both cores hit 90.9 ms independently.
Compute-bound, so bytes don't matter	Doubling the bytes cost 14.6 ms.
MAC16 is the next lever	GCC emits zero m u l a . * either.
Rust is 38% behind C on arithmetic	Assumed C's memory was additive. C's single accumulator stalls and hides it.
120 MHz MSPI is worth ~7 ms	3.4 ms. The head moved 0.3% while its memory floor dropped 13.4 → 9.3 ms.
It looks like a flash-clock win	Flash 40 → 80 MHz measured 106.6 ms — nothing at all.
Four rows per pass will help	Register spill; cycles/MAC 8.13 → 8.41. Predicted in advance, reverted same day.
The board is warming up	A given binary reproduces to 0.01 s. It is code placement.
Host ratio ≈ device ratio, so the host is a proxy	They came apart the moment anything changed — a coincidence mistaken for a method.

Ten entries. Every one killed by a number, usually within a day, usually by an instrument built specifically to kill it.

06 What made all of it possible

The board proves its own output

Every row of the benchmark table before a certain point claimed "byte-identical output" on the strength of a human reading 400 tokens of children's stories and thinking they looked the same. Now the firmware folds every token id it emits into a digest and prints it:

```
token digest: 0x327578cb136fd6aa (418 tokens) OK -- matches models.toml gen_digest
```

The registry pins the value; a host test recomputes it through the same code path. Every experiment since — including both reverted ones — is certified *bit-exact and purely a cost*. Without it, "the int8 head is 14.6 ms slower" would carry an unspoken "...probably while computing the same thing."

Figures are generated, never typed

Every number in the README and the benchmark document comes out of one TOML file through one script — the table, both charts, the headline percentage, even the tok/s in the opening sentence. Before that, the chart was a hand-drawn PNG with no generator: it could not be regenerated, only redrawn, so it silently aged every time anyone reflashd.

We know exactly how well hand-maintenance works, because the one figure the generator could not reach — the tok/s in the README's third paragraph — sat at "9.4" through four re-measurements.

The layout tax

Two commits that do not touch the head moved it 90.3 → 94.6 → 98.0 ms. The first read was thermal drift. Repeat runs from hard reset killed that: a given binary reproduces to 0.01 s of wall clock and 0.1 ms per stage. It is code and `.rodata` placement — the head streams 1.32 MB through a 64 KB cache, and where the loop lands relative to cache lines is worth several percent.

THE SAME PROBE, THE SAME BUFFER, FIVE BINARIES

68 / 72 / 73 / 75 / 81 MB/s. The corollary is uncomfortable: with no run-to-run noise, a difference between two binaries in a stage neither touched is *real* — and is not attributable to the change. Every comparison here is made between builds of the same kind, probe against probe, shipping against shipping.

Even the instrument taxes what it measures. The bandwidth probe runs once at boot, outside the timed loop, and still cost **0.3 ms/token** — purely from fifty lines sitting in the same file and moving everything else. It lives behind a feature flag now.

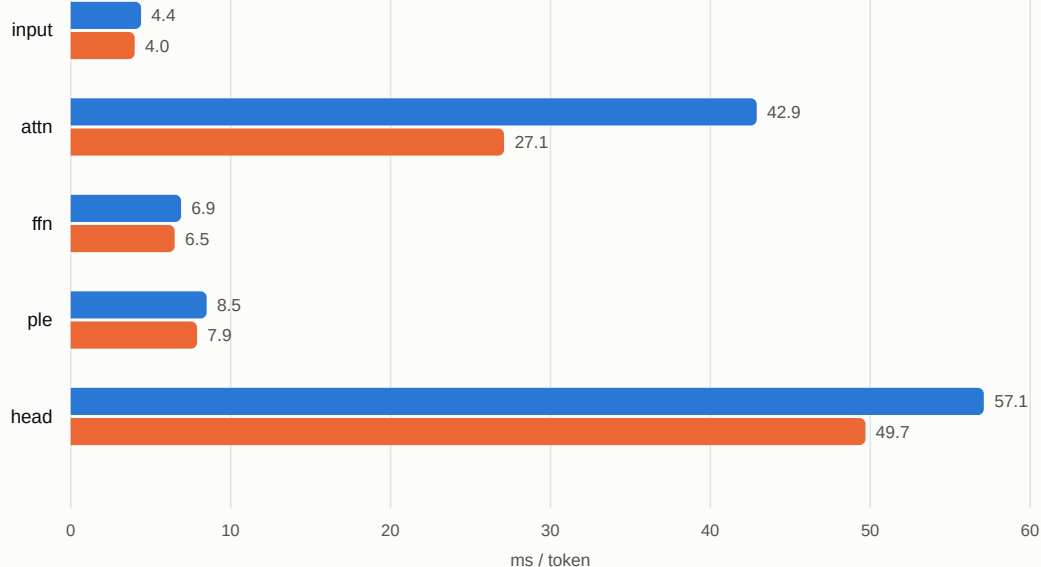
07 Where it ended

Per-stage inference cost, C vs Rust

ESP32-S3 @ 240 MHz · 400 tokens, 18-token prompt · ms per token, lower is better

■ C reference

■ Rust, now



MS / TOKEN	INPUT	ATTN	FFN	PLE	HEAD	WALL	TOK/S
C reference	4.4	42.9	6.9	8.5	57.1	122.0	8.20
Rust	4.0	27.1	6.5	7.9	49.7	95.2	10.50
ratio	0.91×	0.63×	0.94×	0.93×	0.87×	0.78×	

28.2% faster per token, faster on all five stages, byte-identical output. From 311.5 ms at the first working build — a 3.3× improvement over ourselves, and 1.28× over the C we were trying to catch. An opt-in 120 MHz memory-bus build reaches **91.8 ms / 10.89 tok/s** — **32.9% faster than C**; it is not the default, because it needs an experimental ESP-IDF flag and Espressif documents 120 MHz as temperature-sensitive outside the commercial range.

What is left

- **32-bit weight loads** — four bytes per load plus an extract, instead of four single-byte loads. Would require the first `unsafe` in the core crate, for alignment.
- **Four rows at two lanes** — eight live accumulators instead of sixteen, for the same reduction in activation loads that just failed. Cheap to try, genuinely uncertain.
- **fp16 KV cache** — halves what attention streams, and attention is the stage that actually streams. Measured and undecided: it is the only remaining lever that would cost exactness, and this project has not spent any yet.

08 What we'd tell the next person

BUILD THE INSTRUMENT BEFORE THE OPTIMIZATION

Every real win here came from a measurement built to answer one question — cycles/MAC cache-resident versus streaming, both cores reading at once, the four-way attention breakdown. Every wasted day came from reasoning about a number nobody had taken.

A NULL RESULT EXPIRES

Packing the head measured 0.0 ms and was worth 13 ms four changes later. The measurement was correct both times; what changed was everything around it. Nothing in a performance codebase is permanently settled.

WRITE THE FAILURE MODE DOWN BEFORE YOU FLASH

The four-row experiment named its own register-spill risk in the commit message before it was measured. When cycles/MAC went the wrong way there was no argument about what had happened — the diagnosis was already written, and the revert took ten minutes.

KEEP THE NEGATIVE RESULTS IN THE TABLE

Three are visible in the published benchmark today, marked and excluded from the headline. They are the most useful rows there: a plausible idea with a number attached, so the next person to have it finds the measurement instead of spending a day reproducing it.

And check the thing you cannot see

The single most valuable line of output this project produces is a sixteen-digit hexadecimal number that has never once been wrong. **It is what makes every other number in this report mean something.**